

神经网络优化算法：从SGD到Adam

一、SGD

SGD (Stochastic Gradient Descent), 即为随机梯度下降法, 在神经网络中调整参数时使用, 数学表达式为:

$$W \leftarrow W - \eta \frac{\partial L}{\partial W}$$

我们用 Python 代码实现这个类:

```
# lr 是学习率, 其值为手动设定的
class SGD:
    def __init__(self, lr=0.01):
        self.lr = lr

    # params 是传入的权重参数, grads 则为传回的梯度值
    def update(self, params, grads):
        for key in params.keys():
            params[key] -= self.lr * grads[key]
```

简单理解: 梯度在高等数学中是一个指向函数增长最快方向的向量, 因为我们使损失函数的值最小, 因此在这里取它的负值, 则表示函数减小最快的方向, 进而沿着该方向调整参数。

二、Momentum

Momentum, 动量法, 在 SGD 的基础上做了一些改进, 通常有助于减少振荡、加快收敛。同时, 由于动量法引入了惯性, 在接近极小值时可能发生一定程度的越过或振荡, 其数学表达式如下:

$$v \leftarrow \alpha v - \eta \frac{\partial L}{\partial W}$$
$$W \leftarrow W + v$$

重点

由于 SGD 对平坦区域收敛很慢, 类比山谷: 陡坡下降快, 靠近谷底平坦处梯度很小、前进缓慢。因此我们引入新的变量 v , 用来保存之前的运动趋势, 更新参数时保留部分历史方向, 依靠惯性继续前进。

我们用 Python 代码实现这个类:

```
class Momentum:
    # lr 为学习率, momentum 为公式中的  $\alpha$ , 手动设定值
```

```

def __init__(self, lr=0.01, momentum=0.9):
    self.momentum = momentum
    self.lr = lr
    self.v = None

def update(self, params, gradient):
    if self.v is None:
        self.v = dict()
        for key, val in params.items():
            self.v[key] = np.zeros_like(val)

    for key in params.keys():
        self.v[key] = (
            self.momentum * self.v[key]
            - self.lr * gradient[key]
        )
        params[key] = params[key] + self.v[key]

```

三、AdaGrad

AdaGrad 也是一种基于 SGD 的方法，在 SGD 中，所有参数的学习率都是一个相同的值 η ，因此用它解决不同参数适合的学习率可能不同这种问题，其本质就是为每个参数调整合适的学习率，简单来说，如果一个参数过去学得太多，那就在后续学习中让它少学一些，其数学表达式如下：

$$h \leftarrow h + \frac{\partial L}{\partial W} \odot \frac{\partial L}{\partial W}$$

$$W \leftarrow W - \eta \frac{1}{\sqrt{h} + \varepsilon} \frac{\partial L}{\partial W}$$

从数学式中不难看出，AdaGrad 记录了以前梯度的平方和，用以调整有效学习率，简单理解一下，如果过去的 h 越来越大，那么 $\sqrt{h}$ 也会越来越大，而有效学习率则会越来越小，使参数先快点学，再放慢，这种方法可以缓解参数震荡的问题。

我们用 Python 代码简单实现这个类：

```

class AdaGrad:
    def __init__(self, lr=0.01):
        self.lr = lr
        self.h = None

    def update(self, params, gradient):
        if self.h is None:
            # 创建空字典
            self.h = dict()
            for key, val in params.items():
                self.h[key] = np.zeros_like(val)

```

```

for key in params.keys():
    self.h[key] += gradient[key] * gradient[key]
    # 加上一个微小量，防止除数为 0 的情况出现
    params[key] -= (
        self.lr * gradient[key]
        / (np.sqrt(self.h[key]) + 1e-7)
    )

```

但同样，这种方式的缺点也很明显，到训练后期，学习率会越来越小，参数更新越来越慢，出现几乎学不动的情况。

四、RMSProp

RMSProp 是一种基于 AdaGrad 的优化方法，上面我们提到，AdaGrad 的主要问题在于训练后期学习率太小，参数每次几乎不更新。RMSProp 则为梯度和这个值也加上一个参数，即从单纯的梯度平方和变成了梯度平方的加权和，使系统更关注于近期的结果，相较而言轻视一些更早的结果，其数学表达式如下：

$$h \leftarrow \rho h + (1 - \rho) \frac{\partial L}{\partial W} \odot \frac{\partial L}{\partial W}$$

$$W \leftarrow W - \eta \frac{1}{\sqrt{h} + \varepsilon} \odot \frac{\partial L}{\partial W}$$

其中， ρ 就是我们设定的权重系数，它的值通常接近 1，比如 0.9 或者 0.99 这样，直观的能看出来，由于更早的梯度每次都会乘上系数 ρ ，进而它对参数的影响会更小，即所谓的“遗忘”了。

我们用 Python 代码简单实现这个类（与 AdaGrad 基本相同）：

```

class RMSProp:
    def __init__(self, lr=0.01, decay=0.99):
        self.lr = lr
        self.h = None
        self.decay = decay

    def update(self, params, gradient):
        if self.h is None:
            self.h = dict()
            for key, val in params.items():
                self.h[key] = np.zeros_like(val)

        for key in params.keys():
            self.h[key] = (
                self.decay * self.h[key]
                + (1 - self.decay) * gradient[key] * gradient[key]
            )
            params[key] -= (
                self.lr * gradient[key]

```

```
        / (np.sqrt(self.h[key]) + 1e-7)  
    )
```

五、Adam

Adam 是以上方法的集大成者，它相对来说更加通用且好调，在该方法中，引入了前面提到的 Momentum 和 RMSProp，即兼顾了及时调整实际学习率和依照惯性前进的功能，其完整数学表达式如下：

5.1 一阶矩估计

$$m \leftarrow \beta_1 m + (1 - \beta_1) \frac{\partial L}{\partial W}$$

上式中，我们称 m 为一阶矩估计，其主要作用和 Momentum 里的 v 类似，直观上看，它表示的是最近一段时间里，梯度下降的方向，即主要保留梯度最近的方向，其中的参数 β_1 我们通常设置为 0.9。

5.2 二阶矩估计

$$v \leftarrow \beta_2 v + (1 - \beta_2) \left(\frac{\partial L}{\partial W} \odot \frac{\partial L}{\partial W} \right)$$

这个式子中，我们称得到的 v 为二阶矩估计，这里类似于 RMSProp，即代入计算的值是梯度值的平方，消除了正负，只保留了大小，即此处主要保留梯度最近的大小，其中的参数 β_2 我们通常设置为 0.999。

5.3 偏差修正

$$\hat{m} \leftarrow \frac{m}{1 - \beta_1^t}$$
$$\hat{v} \leftarrow \frac{v}{1 - \beta_2^t}$$

上面两个式子是第一次出现，我们称之为偏差修正，其作用是消除初始值对于前几轮训练结果的影响，因为在刚开始训练时，我们会将 m 和 v 的初始值设置为 0，但这样会使得得到的 m 和 v 太小，偏向于 0，偏差修正的作用就是把这个从 0 起步造成的影响除掉。

而当训练轮数很大时， t 的值增大， β_1^t 和 β_2^t 逐渐趋于 0，两个修正式分母趋近于 1，因此偏差修正的影响会逐渐减小。

5.4 参数更新

$$W \leftarrow W - \eta \frac{1}{\sqrt{\hat{v}} + \varepsilon} \odot \hat{m}$$

最后一步即为代入我们从上式中得到的中间量，更新参数，记得分母加上微小量，避免除以零。

5.5 Python 实现

我们已经简单了解了 Adam 的更新逻辑，现在，我们用 Python 代码简单实现这个类：

```
class Adam:
    def __init__(self, lr=0.001, beta1=0.9, beta2=0.999, eps=1e-7):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.eps = eps

        self.m = None
        self.v = None

        # 记录更新次数
        self.t = 0

    def update(self, params, grads):
        # 第一次更新时，为每个参数创建 m 和 v
        if self.m is None:
            self.m = {}
            self.v = {}

        for key in params.keys():
            self.m[key] = np.zeros_like(params[key])
            self.v[key] = np.zeros_like(params[key])

        # 更新次数
        self.t += 1

        for key in params.keys():
            g = grads[key]

            # 更新一阶矩
            self.m[key] = self.beta1 * self.m[key] + (1 - self.beta1) * g

            # 更新二阶矩
            self.v[key] = (
                self.beta2 * self.v[key]
                + (1 - self.beta2) * (g**2)
            )

            # 偏差修正
            m_hat = self.m[key] / (1 - self.beta1**self.t)
            v_hat = self.v[key] / (1 - self.beta2**self.t)

            # 4. 更新参数
            params[key] -= (self.lr * m_hat) / (np.sqrt(v_hat) + self.eps)
```